FINE-TUNING OPEN-SOURCE LLMs WITH QLoRA

Building Your Own Specialized Language Model

CORE CONCEPTS

What Does "Building Your Own Model" Mean?

Reality Check

- ✕ We're **NOT** training from scratch
  Avoids massive compute costs
- ✕ We're **NOT** building a new architecture
  Leverages proven transformer blocks
- ✓ We **ARE** fine-tuning an existing model
  `Adapting weights for specific tasks`

Common Base Models

- **Meta's Llama 3.2** INDUSTRY STANDARD — POPULAR
- Google's Gemma
- Microsoft's Phi
- DeepSeek
- Alibaba's Qwen

STRATEGIC MOTIVATION

Why Fine-Tune Open-Source Models?

Advantages

- ✓ Free to use and deploy
- ✓ Run on your own hardware
- ✓ Full control over the model
- ✓ No API costs
- ✓ Data privacy `(runs locally)`

The Challenge

Frontier Models — 100B+ Params
HUGE RESOURCE DEMAND

VS

Local Models — 3B Params
ACCESSIBLE HARDWARE

REQUIREMENT
Need **efficient** training techniques

MEMORY CONSTRAINTS

The Challenge

Problem: Memory Wall

BASE MODEL
Llama 3.2 (3B) — 13 GB

+

TRAINING OVERHEAD
Requires ~4x Memory
- Forward Pass
- Backward Pass
- Gradients
- Optimizer States

=

TOTAL GPU MEMORY NEEDED
~50 GB+ — EXCEEDS CONSUMER GPU

Solution

QLoRA

Fine-tune huge models on a single GPU

✓ Quantization ✓ Low-Rank Adaptation

→ Drastically reduces memory footprint while maintaining performance.

METHODOLOGY

What is QLoRA?

QLoRA

Quantization
+
LoRA

Two separate tricks combined:

LoRA
LOW-RANK ADAPTATION
Efficiently adapts pre-trained model weights by injecting trainable rank decomposition matrices.

Quantization
PRECISION REDUCTION
Reduces the memory footprint by representing weights with fewer bits (e.g., 4-bit vs 32-bit).

Both techniques reduce memory requirements **dramatically**!

METHODOLOGY

LoRA - The Core Idea

! THE PROBLEM

3 Billion
parameters are too many to train
Requires massive compute

✓ THE RESULT

Train only
18 Million
Instead of 3 Billion!
(>99% parameter reduction)

LoRA Solution Process

1. **Freeze Original Weights**
   Don't train the base model parameters.
2. **Add Adapter Matrices**
   Insert small LoRA matrices into the architecture.
3. **Train Only Small Matrices**
   Update only the adapter weights during backprop.
4. **Add to Frozen Base**
   Combine trained adapters with the original model.

METHODOLOGY

LoRA - How It Works

Architecture Components

BASE MODEL
Llama 3.2
Frozen (3B parameters)

TARGET MODULES
Attention Layers
Q K V O

LORA MATRICES
Matrix A & Matrix B
Low-rank adapter matrices (Trainable)

The Formula

```
Output = Base_Layer + (alpha × LoRA_A × LoRA_B)
```

● Frozen Weights ● Trainable Adapters ● Scaling Factor

KEY INSIGHT
LoRA_A and LoRA_B are much smaller than base layers!
This drastic reduction in trainable parameters is what makes fine-tuning possible on consumer hardware.

CONFIGURATION

LoRA Hyperparameters

config.py

```
peft_config = LoraConfig(
  r=32, // Rank
  lora_alpha=64, // Scaling
  target_modules=[
    "q_proj",
    "v_proj"
  ],
  lora_dropout=0.05
)
```

Typical configuration for 7B+ models

Rank (r) `8, 16, 32, 64`
The number of dimensions in LoRA matrices. Determines model capacity.
↑ Higher r = More capacity, slower training

Alpha (Scaling Factor)
Determines how much weight the adapter updates carry compared to the base model.
Rule of Thumb: alpha = 2 × r

Target Modules
Specific layers within the transformer blocks to attach adapters to.
- ✓ **Start with:** Attention layers (q_proj, k_proj, v_proj, o_proj)
- + **Advanced:** Add MLP layers for more capacity

CONCEPT

Quantization - The Q in QLoRA

PRECISION REDUCTION

32-bit Float
Full Precision
↓
8-bit Int
Quantized
↓
4-bit Int
QLoRA Target

What is Quantization?
Process of reducing the precision of each weight parameter. Moving from high-precision floating point numbers (32-bit) to compact integers (4-bit).

Analogy: The Dimmer Switch

32-bit	8-bit	4-bit
Infinite positions	256 positions	16 positions
(Smooth)	(Fine)	(Coarse)

THE TRADE-OFF
- ✓ **Massive Memory Savings**
  13GB → 2.2GB
- ! Slight Performance Drop
  Negligible for most tasks

EFFICIENCY ANALYSIS

Memory Comparison

CONFIGURATION	MEMORY SIZE
Llama 3.2 (32-bit)	13 GB
Llama 3.2 (8-bit)	3.6 GB
Llama 3.2 (4-bit)	2.2 GB
+ LoRA (r=32)	+73 MB
Total QLoRA	**2.27 GB**

Massive Reduction

By combining 4-bit quantization with LoRA adapters, we reduce memory requirements by **~82%** compared to the full 32-bit model.

✓ HARDWARE ENABLED
You can fine-tune on a free Google Colab T4 GPU!
Requires < 15GB VRAM

MODEL ARCHITECTURE

Base Model vs Chat Model

Base Model

PRIMARY FUNCTION
Predicts the next token
Raw language completion capabilities

INPUT / OUTPUT FLOW
Input → Completion

✓ **Best For**
Specialized tasks with one specific instruction type

VS

Chat / Instruct Model

PRIMARY FUNCTION
Fine-tuned for conversations
Optimized for instruction following

STRUCTURE
- System → *Prompt context*
- User → Message
- Assistant → Response

✓ **Best For**
Interactive applications with varied and complex instructions

For QLoRA: Choose your base model based strictly on your target task!

CONFIGURATION

Training Hyperparameters

config.json

```
"num_epochs":      3
"batch_size":      4
"learning_rate":   2e-4
```

SFT TRAINER CONFIG

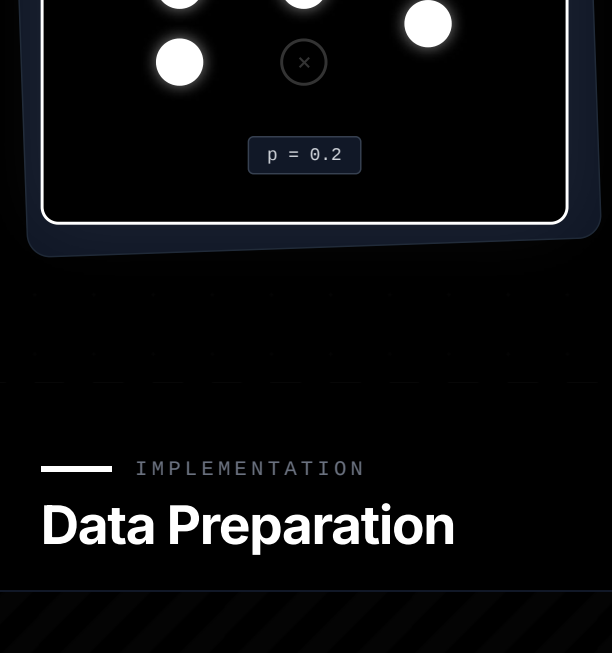
Epochs — Range: 1-3
TRAINING PASSES
Number of complete passes through the dataset. More epochs increase learning but introduce risk of overfitting.

Batch Size — Values: 4, 8, 16
EXAMPLES PER STEP
Determines how many examples are processed at once. **Maximize this** based on your available GPU memory limits.

Learning Rate — Typ: 1e-4 to 5e-4
STEP MAGNITUDE
Controls the size of weight updates. Too high creates instability; too low results in extremely slow convergence.

HYPERPARAMETERS

Dropout



What is Dropout?
Randomly "turn off" **10-20% of neurons** during training. A different set of neurons is deactivated for every training step.

Why Use It?
- Prevents model overfitting
- Forces model to learn general patterns
- Makes model more robust to noise

RECOMMENDED SETTING
0.1 (10%) or **0.2 (20%)** — `dropout_rate = 0.1`

IMPLEMENTATION

Data Preparation

Core Structure

STANDARD FORMAT
Prompt + Completion Pairs

Best Practices

- **Keep prompts consistent**
  Structured inputs help the model learn patterns faster.
- **Limit token length**
  e.g., 110 tokens to optimize memory usage.
- **Round outputs if needed**
  Simpler numbers are easier for small models to predict.

Training Example

JSON / JSONL Format

```
{
  "prompt": "What does this cost to the nearest dollar? \n\n[product description] \n\nPrice is $",
  "completion": "149"
}
```

← INPUT ← TARGET

The model learns to predict the completion text based on the prompt.

WORKFLOW

The Training Process

OVERVIEW

The QLoRA fine-tuning workflow consists of 7 sequential steps, transforming a general-purpose base model into a specialized assistant.

TIME REQUIRED

30 Min to several hours
DEPENDING ON DATA SIZE

1. **Load Base Model**
   Quantized to 4-bit
2. **Add LoRA Adapters**
3. **Prepare Data**
   Prompt-completion pairs
4. **Configure Trainer**
   SFTTrainer settings
5. **Train!**
   Monitor loss actively
6. **Save LoRA Weights**
7. **Evaluate on Test Set**

PROCESS CONTROL

Monitoring Training

Key Metrics

- **Training Loss**
  Should decrease over time
- **Validation Loss**
  Should decrease (watch for overfitting)
- **GPU Memory**
  Should stay under limit

Tools

Tool	Use
TensorBoard	VISUALIZATION
Weights & Biases (wandb)	TRACKING
Hugging Face Hub	MODEL STORAGE

STRATEGIC DECISION MAKING

When to Use QLoRA Fine-Tuning

Good Use Cases

- ✓ Specialized domain knowledge
- ✓ Specific output format
- ✓ Custom tone/style
- ✓ Privacy-sensitive data
- ✓ Cost optimization
  NO API FEES

Not Ideal For

- ✕ General knowledge
  Better to use frontier models
- ✕ Very small datasets
  <100 examples is insufficient
- ✕ Tasks that need reasoning
  Use o1 / o3 models instead

IMPLEMENTATION

Practical Considerations

Resource Planning
Successful fine-tuning requires balancing hardware capabilities, budget constraints, and time allocation.
COMPUTE BUDGET TIME

> *"Start small with free resources, then scale up as requirements grow."*

Hardware Requirements

GOOGLE COLAB	Free T4 GPU (15GB VRAM)
LOCAL	RTX 3090 / 4090 recommended
CLOUD	AWS, GCP, Azure GPU instances

Cost Analysis

ENTRY LEVEL	CLOUD COMPUTE
Free	**$0.50 - $2.00**
Google Colab (limits)	Per hour (approx.)

Time Investment

DATA PREP	TRAINING	EVALUATION
Hours to Days	30m - 4 Hours	Minutes

GUIDELINES

Best Practices

1. **Start Small**
   Use r=8 or r=16 first.
2. **Monitor Overfitting**
   Watch validation loss closely.
3. **Experiment**
   Try different hyperparameters.
4. **Version Control**
   Save models at checkpoints.
5. **Document Everything**
   Track what works and what doesn't.
6. **Test Thoroughly**
   Evaluate on held-out data.

WARNINGS

Common Pitfalls

AVOID
CRITICAL ERRORS

Avoid these mistakes to ensure training stability:

- ✕ **Not enough training data**
  Need 1000+ examples ideally
- ✕ **Too many epochs**
  Risk of overfitting
- ✕ **Wrong base model choice**
  Base vs Instruct
- ✕ **Ignoring data quality**
  Garbage in, garbage out
- ✕ **Not setting max token length**
  Causes OOM errors
- ✕ **Forgetting to quantize**
  Must enable 4-bit loading

IMPLEMENTATION ECOSYSTEM

Tools & Libraries

Essential

Library	Description
`transformers`	Hugging Face library
`peft`	Parameter-Efficient Fine-Tuning
`bitsandbytes`	Quantization
`trl`	Transformer Reinforcement Learning (SFTTrainer)

Optional

Library	Description
`wandb`	Experiment tracking
`tensorboard`	Visualization
`datasets`	Data loading

> *"The `transformers` library is the backbone of modern NLP, while `peft` and `bitsandbytes` enable efficiency."*

INFERENCE PHASE

Deployment Options

After Training

- 01 **Local Deployment**
  Run on your machine directly
- 02 **Cloud API**
  Deploy on AWS, GCP, or Azure
- 03 **Edge Devices**
  Mobile phones & IoT (small models)
- 04 **Hugging Face Spaces**
  Free hosting for demos

Inference

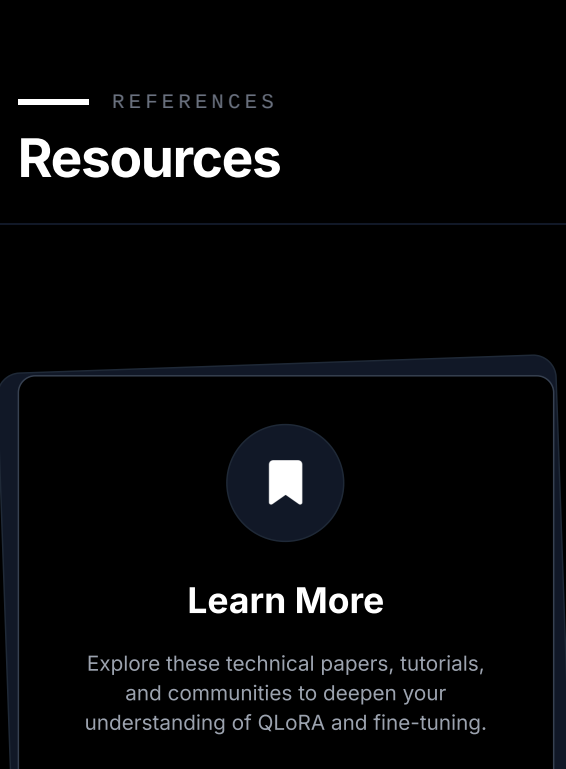
HOW IT LOADS
`Base Model` + `LoRA Weights`
Adapters are loaded dynamically on top of the frozen base model.

Much faster than training
Lightweight forward passes without gradient calculation overhead.

Further Optimization
Can quantize further (e.g., 4-bit, GGUF) for deployment on consumer hardware.

CASE STUDY

Real-World Example



73% Improvement

The Task
Predict product prices accurately based solely on text descriptions.

EXPERIMENTAL SETUP

Base Model	**Dataset**
Llama 3.2 (3B) 4-bit Quantized	20,000 Items Product descriptions
LoRA Config	**Training**
r=32, alpha=64 Attention modules	~30 Minutes 1 Epoch

> *"The fine-tuned model moved from an average error of **$110** down to **$30-40**, demonstrating that small open-source models can match specific domain needs efficiently."*

REFERENCES

Resources

Learn More
Explore these technical papers, tutorials, and communities to deepen your understanding of QLoRA and fine-tuning.

3	2+	3
PAPERS	GUIDES	HUBS

Documentation & Papers
- Hugging Face: `docs/peft`
- QLoRA Paper: `arxiv 2305.14314`
- LoRA Paper: `arxiv 2106.09685`

Tutorials
- Hugging Face Course `/learn`
- Google Colab Examples: Search `"QLoRA fine-tuning"`

Community
`HF Forums` `r/LocalLLaMA` `Discord: HF` `EleutherAI`

SUMMARY

Key Takeaways

CORE CONCEPT

QLoRA =
Quantization + LoRA
Two powerful memory-saving tricks combined to democratize LLM fine-tuning.

HARDWARE

Accessibility
Free to Train
Runs on Google Colab T4

Essential Points

1. **QLoRA Formula**
   Quantization + LoRA techniques combined.
2. **Parameter Efficiency**
   Train 18M parameters instead of 3B.
3. **Memory Usage**
   Use 2.2GB memory instead of 50GB+.
4. **Accessibility**
   Fine-tune on free hardware (Google Colab).
5. **Model Selection**
   Choose base vs instruct model based on task.
6. **Configuration**
   Start with r=32, alpha=64 for attention layers.
7. **Monitoring**
   Monitor training loss and validation loss.
8. **Optimization**
   Experiment with hyperparameters to optimize.

IMPLEMENTATION

Next Steps

REMEMBER

Fine-tuning is iterative.
- Experiment frequently
- Analyze results
- ↑ **Improve continuously**

`Start simple, optimize later.`

To Get Started

1. Set up Google Colab with T4 GPU
2. Install required libraries
3. Load Llama 3.2 (or similar model)
4. Prepare your dataset
5. Configure QLoRA parameters
6. Train and evaluate!

?

QUESTIONS?

Thank you!

READY TO BUILD YOUR OWN SPECIALIZED LLM?

REFERENCES

Additional Resources

Explore these repositories and libraries to start your fine-tuning journey.

- **Llama Models**
  HUGGING FACE
  https://huggingface.co/meta-llama
- **PEFT Library**
  GITHUB
  https://github.com/huggingface/peft
- **TRL Library**
  GITHUB
  https://github.com/huggingface/trl
- **Bits and Bytes**
  GITHUB
  https://github.com/TimDettmers/bitsandbytes
- **Example Notebooks**
  GITHUB / TUTORIALS
  https://github.com/huggingface/peft/tree/main/examples